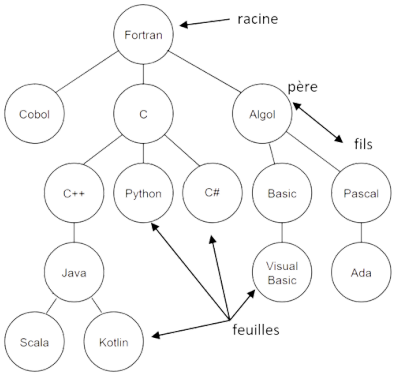
Structures hiérarchiques : arbres

Cours

Un **arbre** est un type abstrait de données constitué d'un ensemble de **nœuds**, reliés entre eux par des **arêtes** et organisés de manière **hiérarchique** :

- Un nœud particulier est la **racine**.
- Chaque nœud peut avoir **aucun, un ou plusieurs fils**. Les nœuds qui n'ont pas de fils sont appelés les **feuilles** de l'arbre, les autres (autre que la racine) sont des nœuds internes. L'**arité** d'un nœud est son nombre de fils.
- Chaque nœud a un **unique père**, à l'exception de la racine qui n'en n'a pas. Les nœuds qui ont le même père sont appelés des **frères**.
- Le **chemin à la racine** d'un nœud est la liste des nœuds qu'il faut parcourir depuis la racine jusqu'au nœud considéré.
- L'**étiquette** est la valeur donnée à chaque nœud (ici les noms de langages informatiques).



Les arbres trouvent de nombreuses applications en informatique, par exemple dans une arborescence de fichiers ou pour des stratégies de jeux.

Cours

- La **taille** d'un arbre est son **nombre de nœuds**.
- La **profondeur** d'un nœud est le **nombre d'arêtes entre la racine le nœud**. La **profondeur de la racine est donc 0**.
- La **hauteur** d'un arbre est la plus **grande profondeur d'une feuille de l'arbre**. **Un arbre réduit à la racine a une hauteur de 0, un arbre vide a une hauteur de -1 (par convention)**.

⚠️ Il n'existe pas de définition universelle pour la hauteur d'un arbre et la profondeur d'un nœud dans un arbre. Dans certains cas la profondeur des nœuds est comptée à partir de 1, la **hauteur de l'arbre réduit à la racine est 1 et la hauteur de l'arbre vide est 0**.

Interface

Les principales primitives constituant l'interface d'un arbrete sont :

- `creer() → arbre` : construire un arbre vide.
- `est_vide() → bool` : vérifier si un arbre est vide ou non.
- `taille() → int` : renvoyer la taille d'un arbre.
- `hauteur() → int` : renvoyer la hauteur d'un arbre.

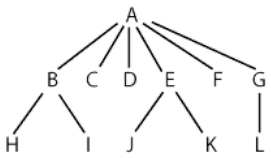
- `profondeur(nœud) → int` : renvoyer la profondeur d'un nœud.
- `tete() → element` : lire le premier élément (la tête) de la liste.
- `queue() → liste` : accéder au reste de la liste (la queue).
- `est_feuille(noeud) → bool` : vérifier si un nœud est une feuille ou pas.
- `branche(noeud) → arbre` : renvoyer un sous-arbre de racine nœud.

Python ne propose pas de façon native l'implémentation des arbres. Ils peuvent être implémentés de plusieurs façons.

Implémentation avec un dictionnaire

Une première implémentation est d'utiliser un dictionnaire dont les clés sont les étiquettes des nœuds et les valeurs des tableaux de fils.

```
arbre = {'A':['B', 'C', 'D', 'E', 'F', 'G'], 'B':['H','I'],
         'C':[], 'D':[], 'E':['J','K'], 'F':[], 'G':['L'],
         'H':[''], 'I':[], 'J':[], 'K':[], 'L':[]
         }
```



Les primitives s'écrivent :

```
def creer():
    return {}

def est_vide(arbre):
    return len(arbre) == 0

def taille(arbre):
    return len(arbre)

def ajouter_noeud(arbre, noeud):
    if noeud not in arbre: arbre[noeud] = []

def ajouter_fils(arbre, noeud, fils):
    ajouter_noeud(arbre, fils)
    arbre[noeud].append(fils)

def est_feuille(arbre, noeud) :
    return len(arbre[noeud]) == 0
```

puis pour créer notre arbre :

```
a = creer()
ajouter_noeud(a, 'A')
ajouter_fils(a, 'A', 'B')
ajouter_fils(a, 'A', 'C')
ajouter_fils(a, 'A', 'D')
# etc.
```

Il est aussi possible de rajouter quelques primitives de profondeur, hauteur, etc. :

```
def pere(arbre, n):
    for noeud in arbre:    # parcourt des noeuds
        if n in arbre[noeud]:
```

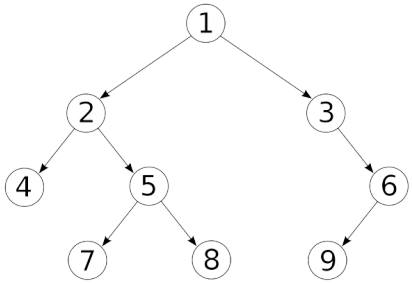
```
            return noeud
    # n n'a pas de pere
    return None

def profondeur(arbre, n):
    p = 0
    while pere(arbre, n) is not None:
        n = pere(arbre, n)
        p = p + 1
    return p

def hauteur(arbre):
    if est_vide(arbre): return -1
    t = []
    for noeud in arbre:
        t.append(profondeur(arbre, noeud))
    return max(t)
```

Arbre binaire

Cours

Un **arbre binaire** (AB) est un cas particuliers d'arbre où **chaque nœud possède au maximum deux fils ordonnés** : un fils **gauche** et/ou un fils **droit**.

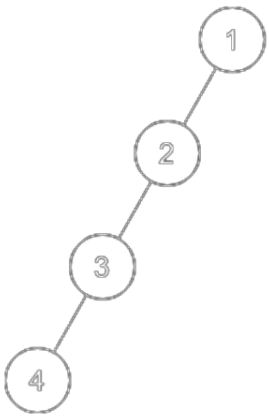
Les fils gauche et droit ne sont pas intervertibles !



Il est possible d'avoir des arbres binaires de même taille mais de « forme » très différente :

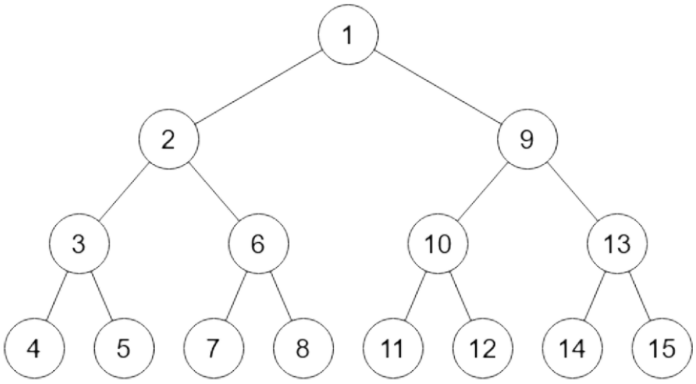
Arbre binaire filiforme (ou dégénéré)

Tous ses nœuds possèdent un unique fils (on parle aussi de peigne).



Arbre binaire parfait

Tous ses nœuds possèdent exactement 2 fils (sauf les feuilles qui en ont zéro !).



Il en résulte certaines propriétés sur la taille n et la hauteur h d'un arbre binaire :

- Un arbre filiforme de taille n a une hauteur h égale à $n-1$, c'est la plus grande hauteur possible donc pour tout AB : $h \leq n–1$.
- On peut aussi montrer[1] qu'un arbre parfait de hauteur h a une taille n égale à $2^{h+1}-1$, c'est la plus grande taille possible donc pour tout AB : $n \leq 2^{h+1}–1$.

Cours

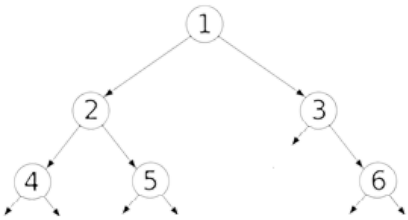
La taille n d'un arbre binaire quelconque de hauteur h est encadrée par : $h+1 \leq n \leq 2^{h+1}-1$

Réciproquement, la hauteur h d'un arbre binaire de la taille n est encadrée par : $log_2(n) \leq h \leq n-1$

Implémentation avec des p-uplets imbriqués

Les arbres binaires ont au plus deux fils, il est donc possible d'utiliser des triplés imbriqués contenant pour chaque nœud : sa valeur, son fils de gauche et son fils de droite (dans cet ordre).

```
>>> n4 = (4, (), ())
>>> n5 = (5, (), ())
>>> n2 = (2, n4, n5)
>>> n6 = (6, (), ())
>>> n3 = (3, (), n6)
>>> n1 = (1, n2, n3)
>>> n1
(1, (2, (4, (), ()), (5, (), ())), (3, (), (6, (), ())))
```

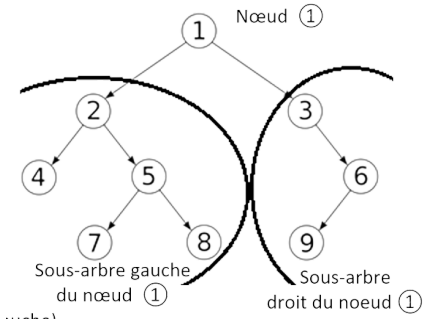


Implémentation récursive

Notons qu'un arbre binaire peut-être est défini de façon récursive comme étant :

- soit un arbre vide,
- soit composé d'un nœud racine avec une valeur, un sous-arbre gauche, et un sous-arbre droit.

Les sous-arbres de gauche et droite sont aussi des arbres (autrement dit un nœud avec deux sous-arbres, etc…).

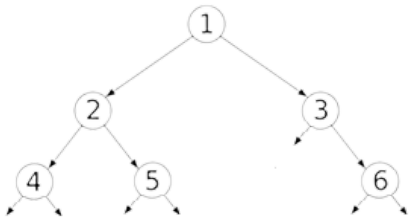


Implémentons sur ce modèle un AB par une classe `Noeud` possédant trois attributs : - `valeur` pour l'étiquette du noeud ; - `gauche`, le sous arbre gauche (ou `None` si le nœud n'a pas de fils gauche) ; - `droite`, le sous arbre droit (ou `None` si le nœud n'a pas de fils droit).

```
class Noeud:
    def __init__(self, v, g=None, d=None):
        self.valeur = v
        self.gauche = g   # None ou un Noeud
        self.droite = d   # None ou un Noeud
```

et créons un arbre non vide (un arbre vide est `None`) :

```
n7 = Noeud(7)
n8 = Noeud(8)
n5 = Noeud(5, n7, n8)
n4 = Noeud(4)
n2 = Noeud(2, n4, n5)
n9 = Noeud(9)
n6 = Noeud(6, n9)
n3 = Noeud(3, None, n6)
a = Noeud(1, n2, n3)
```



ou directement :

```
a = Noeud(1, Noeud(2, Noeud(4), Noeud(5, Noeud(7), Noeud(8))), Noeud(3, None, Noeud(6,
Noeud(9))))
```

Ajoutons une première méthode pour vérifier si un nœud est une feuille :

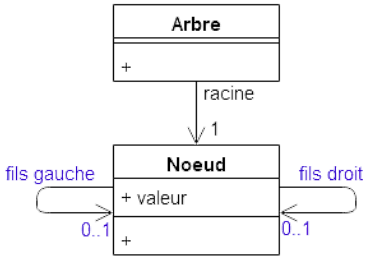
```
    def est_feuille(self) -> bool:
        return self.gauche is None and self.droite is None
```

puis la taille et la hauteur de l'arbre :

```
    def taille(self):
        # taille du sous arbre droit
        if self.droite is None:
            td = 0
        else:
            td = self.droite.taille()

        # taille du sous arbre gauche
        if self.gauche is None:
            tg = 0
        else:
            tg = self.gauche.taille()

        return 1 + td + tg

    def hauteur(self):
        # hauteur du sous arbre droit
        if self.droite is None:
            hd = -1
        else:
            hd = self.droite.hauteur()

        # hauteur du sous arbre gauche
        if self.gauche is None:
```

```
            hg = -1
        else:
            hg = self.gauche.hauteur()

        return 1 + max(hd , hg)
```

On peut reprocher à cette structure de ne représenter correctement que les arbres qui ne sont pas vide et ont une racine -- appelés « arbres **enracinés** », mais pas les arbres vides qui sont représentés par `None`, ce qui n'est pas un objet de la classe `Noeud`.



Une solution est de créer une nouvelle classe d'arbre qui pointe sur la racine quand l'arbre est enraciné ou sur `None` sinon, sur le même modèle des listes chainées avec les classes `Cellules` et `ListeChainees`.

Ajoutons à notre structure cette classe `AB` avec un attribut racine qui est de type `Noeud` ou `None` pour un arbre vide.

```
class AB:
    def __init__(self, racine=None):
        self.racine = racine     # None ou un Noeud
```

Il est maintenant possible d'implémenter un arbre vide comme un objet de la classe `AB` :

```
arbre = AB()
```

un arbre enraciné ainsi :

```
arbre = AB(Noeud(1))
```

et un arbre complet :

```
arbre = AB(Noeud(1, Noeud(2, Noeud(4), Noeud(5, Noeud(7), Noeud(8))), Noeud(3, None, Noeud(6,
Noeud(9)))))
```

Ajoutons les primitives d'un arbre binaire :

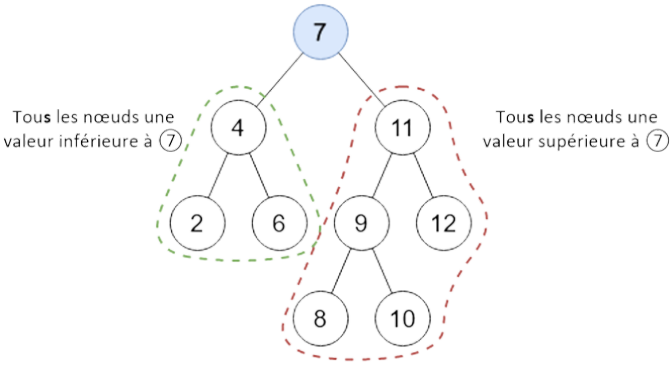
```
class AB:
    def est_vide(self):
        return self.racine == None

    def hauteur(self):
        if self.racine == None: return -1
        # renvoie la hauteur du nœud racine
        return self.racine.hauteur()

    def taille(self):
        if self.racine == None: return 0
        # renvoie la taille du nœud racine
        return self.racine.taille()
```

Arbres binaires de recherche

Cours

Un **arbre binaire de recherche** (ABR) est un cas particulier d'arbre binaire sans lequel :

- Chaque nœud a une valeur (ou clé) supérieure à celles de tous les nœuds de son sous-arbre gauche.
- Chaque nœud a une valeur inférieure à celles de tous les nœuds de son sous-arbre droit.
- Tous les sous arbres sont aussi des ABR



Note : « supérieur » et « inférieur » peuvent être au sens strict ou large en fonction de la définition donnée.

Considèrons l'arbre binaire de recherche précédent qui servira comme support pour illustrer la suite.

Plutôt que de dupliquer la classe `AB` précédente en `ABR` et de la modifier, nous allons créer une sous-classe par héritage[2] et lui ajouter les spécificités d'un ABR.

Inutile de réécrire le constructeur :

```
class ABR(AB):
    pass

a = ABR(Noeud(7, Noeud(4, Noeud(2), Noeud(6)), Noeud(11, Noeud(9, Noeud(8), Noeud(10)),
Noeud(12))))
```

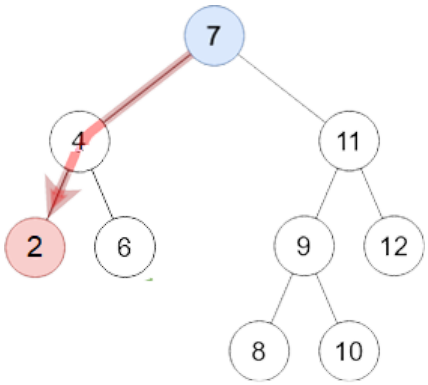
Toutes les méthodes de la classe `AB` fonctionnent par héritage pour un objet de la classe `ABR` :

```
>>> a.taille()
9
>>> a.hauteur()
3
```

Ajoutons des méthodes propres aux ABR :

Clés min et max

Pour accéder à la plus petite clé d'un ABR, il suffit de descendre sur le fils gauche autant que possible. Le dernier nœud visité qui n'a pas de fils gauche porte la plus petite de l'ABR. De la même façon, pour trouver la plus grande valeur il suffit de descendre sur les fils à droite.



La classe `ABR` n'étant pas récursive, il faut définir une **méthode récursive** au niveau de la classe `Noeud` qui descend le plus à gauche[3] :

```
class Noeud:
    def desc_g(self):
        ''' renvoie la feuille la plus à gauche'''
        if self.gauche is None: return self.valeur
        return self.gauche.desc_gauche()
```

puis renvoyer sa valeur dans la classe ABR pour obtenir le min d'un arbre :

```
class ABR :
  def min(self):
      """ Renvoie la plus petite valeur de l'arbre """
      if self.racine is None: return None
      return self.racine.desc_g().valeur
```

Vérifier que l'arbre est un ABR (hors programme)

Pour vérifier qu'un arbre est un ABR, il faut vérifier que :

- La clé de chaque nœud est plus grande que le max de son sous-arbre de gauche, et plus petite que le min de son sous-arbre de droite.
- Les sous-arbres de droites et de gauches sont des ABR.

Implémentons cette vérification de façon récursive au niveau de la classe `Nœud` :

```
class Noeud:

    def verif_noeud(self):
        ''' vérifie que le sous-arbre de racine noeud est un ABR'''
        if self.gauche is None: g = True
        else:
            g = self.valeur > self.gauche.desc_d().valeur and self.gauche.verif_noeud()
        if self.droite is None: d = True
        else:
            d = self.valeur < self.droite.desc_g().valeur and self.droite.verif_noeud()
        return g and d
```

Rajoutons une méthode au niveau de la classe `ABR` :

```
    def verif_ABR(self):
        """ Renvoie True si self est bien un ABR """
        if self.racine is None: return True
        return self.racine.verif_noeud()
```

---

1. Par récurrence la taille d'un arbre racine est $2^0 = 1$, et si la taille d'un arbre parfait de hauteur $h-1$ est $2^h - 1$, pour obtenir la taille de l'arbre parfait de hauteur h il faut ajouter 2^h nouveaux nœuds, au total on obtient $2^h - 1 + 2^h = 2 \times 2^h - 1 = 2^{h+1} - 1$. ↩
2. L'héritage est un des grands principes de la programmation orientée objet (POO) permettant de créer une nouvelle classe à partir d'une classe existante. La sous classe hérite des attributs et des méthodes de la classe mère et en ajoute de nouveaux. ↩
3. On peut aussi définir la sous fonction récursive directement dans la méthode `min()` de la classe `ABR`.

```
class ABR
    def min(self):
        """ Renvoie la plus petite valeur de l'arbre """
        def desc_g(n):
            while n.gauche is not None:
            return desc_g(n.gauche)
            return n
        return desc_g(self.racine).valeur
```

↵